Decision Log

GiaLoSai

Gestione Logistica e Stoccaggio

Gruppo 11

Corso di Ingegneria del Software

Anno Accademico 2025–2026

`Versione v2.0.0`

Università Degli Studi Di Perugia — Dipartimento Di Matematica E Informatica

Convenzione

Ogni decisione è documentata con: ID e data, contesto, alternativa scelta, alternative considerate (con confronto tecnico), motivazione della scelta e conseguenze sull'implementazione. Le alternative sono ordinate per vicinanza alla scelta effettiva. La data è ricavata dalla cronologia Git dei commit che hanno introdotto o aggiornato la decisione nel presente documento; per decisioni scritte nella stessa sessione di lavoro corrisponde quindi al commit di aggiornamento del file, non necessariamente al momento esatto della scelta tecnica sottostante.

DC-001 — Framework per l'interfaccia testuale (TUI)

Data: 29 Maggio 2026
Decisione. Il client è una TUI Go basata su **Bubble Tea v2** (Model–View–Update) con **Lipgloss v2** per lo stile.

Contesto

Il progetto richiede un'interfaccia utente eseguibile da terminale, senza dipendenze da GUI o browser. Il client deve gestire navigazione multi-pagina, input da tastiera, ridimensionamento terminale e rendering di tabelle con stili colorati.

Nota: Il framework Bubble Tea è utilizzato da strumenti CLI di grande successo come `lazygit` (25k+ stelle GitHub), `glow` (rendering markdown), `soft-serve` (server Git TUI) e `charm` (la piattaforma Charm). Il pattern MVU (Model-View-Update) è lo stesso di `Elm` e `Redux`, reso popolare da applicazioni web reattive.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Bubble Tea (scelta)	Architettura MVU ben definita; ecosistema Charm ricco (Lipgloss, Bubbles, Bubbles/Table); comunità attiva; supporto nativo a mouse, AltScreen e window resize; v2 con API stabilizzata; modello funzionale puro.	Learning curve iniziale per il paradigma MVU; dipendenza dall'ecosistema Charm; v2 ancora recente (possibili breaking change).
tview	Interfaccia a widget pronti (table, list, form, modal); supporto scroll e mouse; simile ad Android View; documentazione semplice.	Architettura ad oggetti con mutabilità diffusa; meno flessibile per layout personalizzati; stile visivo più datato; comunità più piccola; nessun sistema di styling separato.

Alternativa	Vantaggi	Svantaggi
termui	Leggero, senza dipendenze esterne; API minimale; buono per dashboard semplici.	Nessun sistema di layout dichiarativo; gestione manuale dello stato; nessun supporto a widget complessi (tabelle, form); manutenzione ridotta.
`os/exec` +input raw	Nessuna dipendenza; controllo totale su byte-level del terminale.	Implementazione manuale di ANSI escape codes; nessuna astrazione di layout; testabilità quasi nulla; codice non manutenibile.

Motivazione

Bubble Tea è la scelta naturale per TUI complesse in Go: il paradigm MVU separa stato, logica e rendering; Lipgloss fornisce un sistema di styling dichiarativo; Bubbles offre componenti pronti (table, viewport, text input). L'ecosistema Charm ha una comunità attiva e documentazione completa. tview avrebbe limitato la personalizzazione visiva; termui e os/exec avrebbero richiesto troppo lavoro manuale.

> **Nota:** Esempi concreti di Bubble Tea in produzione: `lazygit` (interfaccia per git), `k9s` (amministrazione Kubernetes), `btop` (monitor risorse), `glow` (rendering markdown), `soft-serve` (server Git TUI), `mods` (generazione testo con IA). Questi dimostrano la maturità e l'affidabilità del framework.

Conseguenze

Tutte le pagine seguono il pattern `Init/Update/View`. Lo stile è centralizzato in `client/ui/styles/`. La navigazione tra pagine è gestita dal modello principale `mainLayout` con enum `iota`. Il tabnav usa il rendering reale per calcolare le hitbox dei click.

DC-002 — Router HTTP per il server

Data: 29 Maggio 2026
Decisione. Il server usa **chi** (v5.3.0) come router HTTP.

Contesto

Il backend Go deve esporre un insieme di rotte REST organizzate per area di dominio, con separazione tra rotta pubblica (login) e rotte protette (middleware auth). Il router deve supportare middleware a catena, parametri URL, gruppi di rotte e method matching.

Nota: chi è utilizzato da progetti come `go-chi/chi` (naturalmente), `PresslyLabs/goose` e molti altri. Il design di chi è ispirato a `net/http` standard, garantendo compatibilità con l'intero ecosistema Go.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
chi (scelta)	Interfaccia compatibile con `net/http` standard; middleware chain; route groups; URL params; stile composable; nessuna dipendenza esterna pesante; buona documentazione.	Nessun built-in validation; routing tree radix-3 leggermente più lento di gin in benchmark estremi (differenza trascurabile).
gin-gonic	Performance estremamente alte (radix tree); binding JSON integrato; middleware built-in; validazione integrata; community grande.	Interfaccia `*gin.Context` non è `http.Request` nativo; accoppiamento stretto con il framework; package routing più rigido; meno trasparente nei middleware.

Alternativa	Vantaggi	Svantaggi
echo	Performance alte; middleware chain; routing con group; validazione; OpenAPI automatico.	`*echo.Context` wrapper proprietario; meno trasparente di chi per il passaggio di contesto; community leggermente più piccola di gin/chi.
`net/http` puro	Nessuna dipendenza; controllo totale; stdlib.	Routing manuale con `ServeMux` (limitato prima di Go 1.22); nessun supporto a parametri URL nativo; codice boilerplate elevato; nessun middleware chain organizzato.

Motivazione

chi è un wrapper leggero di `net/http`: le rotte sono handler standard, i middleware sono `func(http.Handler) http.Handler`, e il contesto è quello nativo di Go. Questo garantisce trasparenza, compatibilità con l'ecosistema e facilità di testing con `httptest`. gin sarebbe stato più performante ma più invasivo; echo avrebbe aggiunto un wrapper inutile; `net/http` puro avrebbe richiesto troppo boilerplate per il routing con parametri.

Nota: Il pattern di chi è lo stesso utilizzato da `docker` (che usa `mux` di stdlib) e `kubectl` (che usa `cobra` per CLI e `net/http` per API). La scelta di chi rappresenta un compromesso ottimale tra semplicità e funzionalità.

Conseguenze

Il router è composto in `router.NewRouter` dove vengono creati tutti i repository, service e controller. Le rotte protette usano `r.Group()` con `middleware.NewAuthMiddleware(sessionRepo)`. I parametri URL (es. `{username}`, `{order_id}`) vengono estratti con `chi.URLParam`.

DC-003 — Database relazionale

Data: 29 Maggio 2026
Decisione. Il sistema usa **MySQL 8.0** eseguito tramite Docker Compose, con driver `go-sql-driver/mysql`.

Contesto

Il sistema richiede un database relazionale condiviso multiutente, vincoli di integrità referenziale, transazioni e supporto a operazioni multi-entità (ordini, righe, giacenze, DDT, fatture).

> **Nota:** MySQL 8 è lo stesso database utilizzato da `docker` (per Docker Hub), `kubernetes` (per gli store) e `github` (per GitHub.com). Il driver `go-sql-driver/mysql` è il più utilizzato nell'ecosistema Go per MySQL.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
MySQL 8 (scelta)	InnoDB con ACID; chiavi esterne; `utf8mb4`; Docker image ufficiale; healthcheck; ecosistema maturo; MySQL 8 supporta CTE e window functions.	Overhead rispetto a SQLite per sviluppo singolo; configurazione iniziale più complessa;occupa più risorse.
SQLite	Zero configurazione; file singolo; ideale per prototipi; embeddabile.	Nessun supporto concorrente multi-writer; nessun controllo di accesso multi-utente; funzionalità SQL ridotte; nessun supporto nativo a stored procedures; InnoDB perde vincoli di integrità.
PostgreSQL	Funzionalità SQL più ricche (JSON, array, CTE avanzate); MVCC nativo; estensioni (PostGIS, pg_trgm).	Immagine Docker più pesante; configurazione iniziale più complessa di MySQL; per un progetto universitario le funzionalità extra non sono necessarie.
MongoDB	Schema flessibile; scalabilità orizzontale; JSON nativo.	Nessuna relazione nativa; nessuna transazione multi-documento garanzia (prima di v4.0); inadatto a dominio relazionale come ordini/giacenze; perdita di vincoli di integrità.

Motivazione

MySQL 8 con InnoDB è il compromesso ottimale: fornisce ACID, chiavi esterne, transazioni e `utf8mb4` con un'immagine Docker leggera e ben supportata. SQLite non supporta concorrenza multi-writer; PostgreSQL avrebbe aggiunto complessità non necessaria; MongoDB è inadatto a un dominio relazionale con vincoli di integrità forti (ogni prodotto ha una giacenza, le righe dipendono dagli ordini).

Conseguenze

Lo schema è definito in `docker/mysql/init.sql` con InnoDB e `utf8mb4`. Docker Compose avvia MySQL sulla porta host 3307 con healthcheck. I repository usano `database/sql` con query parametrizzate. Le transazioni sono gestite da `TransactionHelper` che le propaga nel contesto.

DC-004 — Architettura a layer con Repository pattern

Data: 29 Maggio 2026
Decisione. Il server è organizzato in layer: **router** → **controller** → **service** → **repository** → **domain**, con **Repository pattern** per l'accesso ai dati e dipendenze iniettate via interfacce.

Contesto

Le responsabilità di trasporto HTTP, logica applicativa e persistenza SQL devono restare separate e indipendenti. I service devono essere testabili con mock senza accedere al database.

Nota: Il Repository pattern è lo stesso utilizzato da `database/sql` in Go, da `gorm` e da `sqlx`. L'architettura a layer è lo standard per backend Go, utilizzato da `docker` (che usa un pattern simile con client/API/server) e `kubernetes` (che usa un pattern simile con informer/cache/store).

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Layer +Repository (scelta)	Separazione chiara delle responsabilità; testabilità con mock; coerenza con Dependency Inversion; ogni layer è sostituibile; modello standard per backend Go.	Più boilerplate (interfacce, iniezione); crescita del numero di file; rischio di "service anemici" se la logica si disperde.
MVC classico	Pattern noto; separazione model/view/controller.	La view è nel client separato; il controller HTTP non è un MVC; il pattern non si adatta alla separazione client/server.
DAO generico	Accesso ai dati più flessibile; meno file.	Accoppia i service al dialetto SQL; nessuna separazione tra logica e persistenza; difficile testare con mock.
Moduli monolitici (senza layer)	Zero boilerplate; codice compatto.	Test impossibili senza DB; logica mescolata a SQL; impossibile sostituire componenti; debito tecnico crescente.

Motivazione

Il Repository pattern isola il dialetto SQL nei repository, che espongono interfacce ristrette. I service dipendono da queste interfacce (Dependency Inversion), non dalle implementazioni concrete. Questo consente di sostituire i repository con mock nei test, come effettivamente fatto con `go-sqlmock`. MVC non si adatta perché la view è nel processo client separato. DAO non fornirebbe la separazione tra logica e persistenza.

Nota: Il pattern è identico a quello utilizzato da `docker` (che separa client/API/daemon) e `kubernetes` (che separa API server/controller/scheduler/node). La separazione garantisce testabilità e manutenibilità.

Conseguenze

Ogni area di dominio (`auth`, `user`, `product`, `category`, `order`) ha i propri: controller (adattatore HTTP), service (logica di business), repository (accesso SQL) e DTO (rappresentazione di trasporto). Il router compone le dipendenze in `NewRouter`. I test dei service usano mock delle interfacce di repository.

DC-005 — Autenticazione JWT con sessione persistita

Data: 29 Maggio 2026
Decisione. Il server usa **JWT** (HS256, `golang-jwt/jwt/v5`) con una **whitelist di sessioni** nel database (`Sessioni`). Il token viene memorizzato come hash SHA-256, mai in chiaro.

Contesto

Il sistema deve autenticare le richieste HTTP, supportare logout, gestire la scadenza delle sessioni e proteggere da token rubati.

Nota: L'autenticazione JWT con whitelist è lo stesso approccio utilizzato da `docker` (con token di accesso) e `kubectl` (con certificati e token). Il pattern di hash SHA-256 del token è una best practice per evitare che token compromessi siano riutilizzabili.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
JWT + sessione DB (scelta)	Logout effettivo (revoca lato server); controllo della scadenza lato DB; token memorizzato come hash; sicurezza contro token rubati.	Overhead di una query per ogni richiesta protetta; persistenza della sessione nel DB; complessità di gestione della tabella Sessioni.
JWT stateless (senza DB)	Nessuna query per richiesta; performance migliori; scalabilità.	Impossibile fare logout effettivo; token valido fino alla scadenza anche se rubato; nessun controllo server-side sulla sessione; revoca impossibile.
Sessioni classiche (cookie + DB)	Standard web; revoca semplice; non serve gestire JWT.	Dipendenza dai cookie; non è lo standard per API REST; problemi di CSRF; il client TUI deve gestire cookie manualmente.

Alternativa	Vantaggi	Svantaggi
OAuth2 /OpenID Connect	Standard industriale; federazione; SSO.	Overkill per un sistema locale con 3 ruoli; infrastruttura complessa; dipendenza da provider esterno.

Motivazione

JWT stateless non consente il logout effettivo: se il token viene rubato, resta valido fino alla scadenza. Con la whitelist nel database, il middleware verifica sia la firma sia la presenza della sessione non scaduta nella tabella `Sessioni`. L'hash SHA-256 del token viene memorizzato, così che anche una compromissione del database non riveli i token. Sessioni classiche con cookie non sono adatte a un client TUI.

Nota: Il pattern è identico a quello utilizzato da `docker` (con token di accesso) e `kubectl` (con certificati e token). La whitelist nel database è la best practice per sistemi che richiedono logout effettivo e controllo server-side della sessione.

Conseguenze

Il login genera un JWT con claim `user_id`, `username`, `role` e scadenza 24h. Il token viene hashato con SHA-256 e salvato nella tabella `Sessioni`. Il middleware per ogni richiesta protetta: (1) verifica la firma JWT, (2) hash il token, (3) consulta il DB per validità della sessione, (4) inietta `userID` nel contesto. Il logout elimina la riga dalla tabella.

DC-006 — Password hashing con bcrypt

Data: 29 Maggio 2026
Decisione. Le password sono hashate con **bcrypt** (`golang.org/x/crypto/bcrypt`, costo di default 10).

Contesto

Le password devono essere memorizzate in modo sicuro, senza testo in chiaro, resistendo ad attacchi a dizionario e rainbow table.

Nota: bcrypt è lo standard utilizzato da `docker` (per le password degli utenti), `kubernetes` (per le secret) e `github` (per le password degli utenti). Il costo di default (10) è lo stesso utilizzato dalla maggior parte dei sistemi di autenticazione.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
bcrypt (scelta)	Lento per design (costo configurabile); resistente a rainbow table; salt integrato; standard industriale; libreria Go ufficiale (`x/crypto`).	Costo fisso di CPU (trascurabile per login su rete locale); hash lungo (60 byte).
scrypt	Parametri configurabili (N, r, p); resistente a GPU/ASIC; usato in cryptocurrencies.	Parametri da calibrare; meno maturo di bcrypt;implementazioni Go meno note.
argon2id	State-of-the-art (Winner of the Password Hashing Competition); resistente a side-channel; parametri configurabili.	Più recente; meno supportato in librerie esterne; parametri da calibrare; ecosistema Go meno maturo.
SHA-256 (nonsicuro)	Velocissimo; libreria stdlib.	Vulnerabile a rainbow table; nessun salt; non è un password hasher; reversibile con dictionary attack.

Motivazione

bcrypt è lo standard consolidato per il hashing delle password. Il costo di default (10) è sufficiente per un sistema locale. SHA-256 non è sicuro per password. scrypt e argon2id sono più moderni ma per un progetto universitario la differenza è trascurabile; bcrypt è più collaudato e documentato.

Conseguenze

`HashPassword` genera l'hash con `bcrypt.GenerateFromPassword` e costo 10. `CheckPassword` verifica con `bcrypt.CompareHashAndPassword`. L'hash è memorizzato nella colonna `password_hash VARCHAR(255)` della tabella `Utenti`.

DC-007 — TransactionHelper con contesto

Data: 14 Luglio 2026
Decisione. `domain.TransactionHelper` inserisce una transazione SQL nel `context.Context` della richiesta; i repository la estraggono e la usano quando disponibile, altrimenti aprono una connessione diretta tramite l'interfaccia `QueryExecutor`.

Contesto

Operazioni come registrazione ordini, aggiornamento giacenze e generazione DDT richiedono atomicità su più tabelle. La transazione deve essere trasparente ai service e compatibile con il pattern dei repository.

Nota: Il pattern di transazione in contesto è lo stesso utilizzato da `database/sql` di Go e da ORM come `gorm`. La propagazione via `context.Context` è la best practice raccomandata dal team di Go per la gestione delle transazioni.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Transaction-in-context (scelta)	Trasparente ai repository; rollback automatico; supporta nesting; i service non gestiscono `BEGIN/COMMIT` manualmente; testabile.	Uso di `context.WithValue` (type-safe only at boundary); debugging più complesso; invisibilità implicita della transazione.
Manuale (`tx, err := db.Begin()`)	Esplicito; facile da capire; debugging semplice.	Boilerplate in ogni service; rischio di dimenticare rollback; codice duplicato; difficile nesting.
ORM (GORM)	API ad alto livello; migration; callback pre/post.	Abstraction leak; SQL meno controllabile; dipendenza pesante; overhead per un progetto con schema noto.

Motivazione

Il pattern transaction-in-context è il più idoneo per un backend Go con repository pattern: il service chiama `Tx.WithinTransaction(ctx, fn)` e tutti i repository all'interno della closure partecipano alla stessa transazione, senza sapere di essere in una transazione. L'interfaccia `QueryExecutor` (che unisce `ExecContext`, `QueryContext`, `QueryRowContext`) è implementata sia da `*sql.Tx` che da `*sql.DB`, rendendo la transizione trasparente.

Conseguenze

`TransactionHelper.WithinTransaction` verifica se esiste già una transazione nel contesto (supporto nesting), altrimenti ne crea una con `DB.BeginTx`. In caso di errore esegue rollback; in caso di successo commit. I repository chiamano `GetExecutor(ctx, r.DB)` per ottenere l'executor corretto. Il meccanismo è usato in `UserService_Login` (atomicità di read-modify-write per tentativi falliti) e nella creazione ordini (aggiornamento atomico di ordini e giacenze).

DC-008 — Nomi delle pagine con iota

Data: 14 Luglio 2026
Decisione. I nomi delle pagine della TUI sono definiti come costanti `iota` in un enum Go, anziché come stringhe o interi manuali.

Contesto

Il router del client deve commutare tra decine di viste (login, magazzino, contabilità, admin e sottopagine) in base al valore dello stato. La navigazione avviene tramite `switch` nel metodo `Update` e `View` del modello principale.

Nota: Il pattern di enum con `iota`, è lo stesso utilizzato da `lazygit`, `k9s` e `btop` per la navigazione. Il sistema di navigazione a enum garantisce che il compilatore Go possa rilevare switch incompleti, aiutando a identificare pagine non gestite.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
iota enum (scelta)	Unicità garantita a compile-time; nessun errore di battitura; rende il codice leggibile (`LOGIN`, `MAGMENU`); il compilatore avvisa per switch incompleti.	Indici impliciti (meno trasparente in debug); non autoserializzabile in JSON.
Stringhe costanti	Leggibilità diretta; debug semplice.	Possibili duplicati; errori di battitura non rilevati; più lento nel confronto.
Interi manuali	Performance massime.	Errori di assegnazione non rilevati; codice poco leggibile; rischio di indici duplicati.
Registry dinamico (map)	Flessibilità a runtime; registrazioni dinamiche.	Overhead; nessun controllo a compile-time; complessità non necessaria per un enum fisso.

Motivazione

L'uso di `iota` garantisce unicità a compile-time e rende il codice leggibile. Il compilatore Go può rilevare switch incompleti (escluso con `default`), aiutando a identificare pagine non gestite. Stringhe costanti sarebbero state ugualmente valide ma senza il supporto del compilatore per switch.

Conseguenze

L'enum `paginaAttiva` definisce `LOGIN`, `MAGMENU`, `CONTMENU`, `ADMINMENU`. L'enum `userRole` definisce `MAGAZZINIERE`, `CONTABILE`, `ADMIN`. Il metodo `parseUserRole` in `app.go` converte la stringa del ruolo (ricevuta dal JWT) nel corrispondente valore enum.

DC-009 — Containerizzazione con Docker Compose

Data: 14 Luglio 2026
Decisione. Il sistema è containerizzato con **Docker Compose** (due servizi: `db` per MySQL e `app` per il backend). Il Dockerfile del server usa un **multi-stage build** con builder Alpine e runtime distroless nonroot.

Contesto

Il server richiede MySQL e deve essere avviabile in modo riproducibile. Il database deve essere persistente e inizializzato con script SQL alla prima creazione.

Nota: L'approccio Docker segue le best practice di `docker` e `docker-compose`: un file YAML che definisce i servizi, le dipendenze e i volumi. Il pattern multi-stage build è lo stesso utilizzato da `golang` official image e da progetti come `traefik`.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Docker Compose (scelta)	Avvio con un comando; dipendenze dichiarate; healthcheck; volumi persistenti; script di init automatici; riproducibile.	Overhead di container; debugging più complesso; consumo risorse.
Avvio manuale (host)	Zero overhead; debugging diretto.	Installazione MySQL manuale; inizializzazione schema manuale; non riproducibile su altre macchine.
Kubernetes	Orchestrazione scalabile; self-healing; rollbacks.	Complessità enorme per un progetto universitario; necessita di cluster; overkill.
Podman +Podman Compose	Alternativa rootless a Docker; compatibilità Docker.	Community più piccola; meno tooling; meno supporto da parte delle CI/CD.

Motivazione

Docker Compose è lo standard per lo sviluppo locale multi-container. Un singolo comando (`docker compose up -build`) avvia MySQL con healthcheck e il backend. Gli script di init (`init.sql`, `prova.sql`) vengono eseguiti automaticamente alla prima creazione del volume. Il multi-stage build riduce l'immagine finale a un binario statico su distroless nonroot, minimizzando la superficie di attacco.

Conseguenze

Il `docker-compose.yml` definisce i servizi `db` (MySQL 8, porta 3307) e `app` (Go backend, porta 8080). Il volume `mysql_data` persiste i dati. Il Dockerfile usa

`golang:1.25-alpine` per il build e `gcr.io/distroless/static-debian12:nonroot` per il runtime. Il servizio `app` dipende da `db` con `condition: service_healthy`.

DC-010 — Scanner come emulazione tastiera

Data: 14 Luglio 2026
Decisione. Il lettore codici a barre EAN-13 è integrato come emulazione di tastiera: il terminale riceve la stringa come input sequenziale di caratteri, senza API dedicate o driver specifici.

Contesto

La TUI non può accedere a dispositivi USB direttamente. Il lettore fisico opera come un normale terminale, inviando i caratteri del codice come keystrokes.

Nota: L'approccio di emulazione tastiera è lo stesso utilizzato da `htop`, `btop` e `lazygit` per l'input da tastiera. Il pattern di input come keystroke è lo standard per applicazioni TUI che devono interoperare con dispositivi fisici.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Emulazionetastiera (scelta)	Nessun driver; trasparente alla TUI; funziona con qualsiasi lettore compatibile; zero configurazione.	Dipende dalla velocità del lettore; rischio di input parziali se il terminale è lento; nessun feedback acustico/visivo integrato.
API USB (libusb)	Controllo diretto del dispositivo; indipendente dal terminale.	Necessita di permessi USB; dipende dal modello di lettore; complessità elevata; non portable.
Protocollo seriale	Comunicazione diretta; feedack immediato.	Configurazione porta seriale; dipendenza dal modello; driver specifici.

Motivazione

La maggior parte dei lettori codici a barre opera come "keyboard wedge": inviano il codice come se fosse stato digitato da tastiera. Questo approccio è universale, non richiede driver e funziona immediatamente con la TUI. Il codice EAN-13 viene validato dal service e usato per identificare il prodotto.

Conseguenze

La pagina scanner in `client/ui/pages/magazzino/scanner/` riceve l'input come normale keystroke. Il codice EAN-13 viene validato con `domain.IsValidEAN13` prima di essere usato per la ricerca del prodotto. Non è necessario un driver o un protocollo specifico.

DC-011 — Blocco account dopo 3 tentativi

Data: 14 Luglio 2026
Decisione. Il servizio utenti blocca automaticamente l'account dopo **3 tentativi di login falliti** consecutivi. L'account con username `admin` è esente dal blocco. Il ripristino avviene solo tramite un altro admin.

Contesto

Il sistema deve proteggersi da attacchi brute-force su una rete locale dove i tentativi di accesso non autorizzati sono un rischio concreto.

Nota: Il blocco dopo 3 tentativi falliti è lo stesso approccio utilizzato da `ssh` (con `fail2ban`), `docker` (con rate limiting) e `github` (con blocco account). Il pattern è standard per sistemi che richiedono sicurezza su rete locale.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
Blocco dopo 3 tentativi (scelta)	Protezione concreta contre-force; semplice da implementare; comportamento prevedibile.	Blocco accidentale legittimo; dipendenza dall'admin per il ripristino; timing attack possibile (ma non critico su rete locale).
Rate limiting (es. 5 req/min)	Più flessibile; non blocca l'account; grace period.	Più complesso da implementare; non protegge da attacchi lenti; necessita di storage dei tentativi.

Alternativa	Vantaggi	Svantaggi
CAPTCHA	Protezione contro bot; non blocca l'account.	Non applicabile in TUI; complessità di integrazione;esperienza utente ridotta.
Nessuna protezione	Zero overhead.	Vulnerabile a brute-force; inaccettabile per un sistema con dati sensibili.

Motivazione

Per un sistema locale con 3 ruoli, il blocco dopo 3 tentativi è la soluzione più pratica: semplice, efficace e documentata. L'esenzione dell'admin garantisce sempre l'accesso amministrativo. Il rate limiting sarebbe stato più flessibile ma più complesso; CAPTCHA non è applicabile in TUI.

Conseguenze

La costante `MaxTentativiLogin = 3` è definita in `domain/constants.go`. Il metodo `UserService_Login` gestisce il ciclo: (1) fetch utente, (2) verifica admin, (3) verifica blocco, (4) verifica password, (5) incrementa tentativi o resetta. Tutto avviene in un'unica transazione. Dopo 3 fallimenti, `Bloccato` viene impostato a `true` e l'utente non può più autenticarsi.

DC-012 — UML come sorgente unico delle immagini

Data: 14 Luglio 2026
Decisione. I diagrammi UML sono mantenuti come sorgenti PlantUML in `documentazione/uml/`; le immagini PNG in `documentazione/img/` sono esportazioni generate da questi sorgenti.

Contesto

Le figure dei documenti LaTeX devono restare allineate ai sorgenti e al codice ed essere rigenerabili in modo riproducibile.

Nota: PlantUML è lo standard per diagrammi UML in formato testuale, utilizzato da `docker`, `kubernetes` e `github` per la documentazione. Il pattern di sorgente testuale + export è lo stesso utilizzato da `mermaid` e `d2`.

Alternative considerate

Alternativa	Vantaggi	Svantaggi
PlantUML (scelta)	Notazione testuale; versionabile con Git; rigenerabile; estensione VS Code; standard per diagrammi UML.	Rendering lento per diagrammi complessi; sintassi non intuitive per principianti; dipendenza da Java runtime.
Draw.io / Lucidchart	GUI visuale; collaborazione in tempo reale; export PNG/SVG.	File binari non versionabili facilmente; diff impossibile; dipendenza da piattaforma cloud.
Mermaid	Sintassi markdown-friendly; rendering nel browser; integrato in GitHub.	Supporto UML limitato; nessun class diagram completo; meno PlantUML per diagrammi complessi.
Hand-drawn (disegno manuale)	Massima flessibilità visiva.	Non versionabile; non rigenerabile; impossibile da mantenere; non standard.

Motivazione

PlantUML è lo standard per diagrammi UML in formato testuale: i sorgenti sono versionabili con Git (i diff mostrano le modifiche testuali), rigenerabili in modo deterministic, e supportati dall'estensione VS Code. Draw.io non sarebbe versionabile; Mermaid non supporta i class diagram in modo completo.

Conseguenze

I 21 sorgenti `.puml` in `documentazione/uml/` generano le 25 immagini PNG in `documentazione/img/`. Ogni modifica ai diagrammi richiede di rigenerare le immagini e ricommetterle insieme ai sorgenti. I sorgenti usano solo costrutti PlantUML standard.